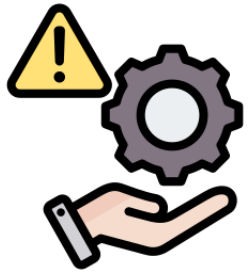
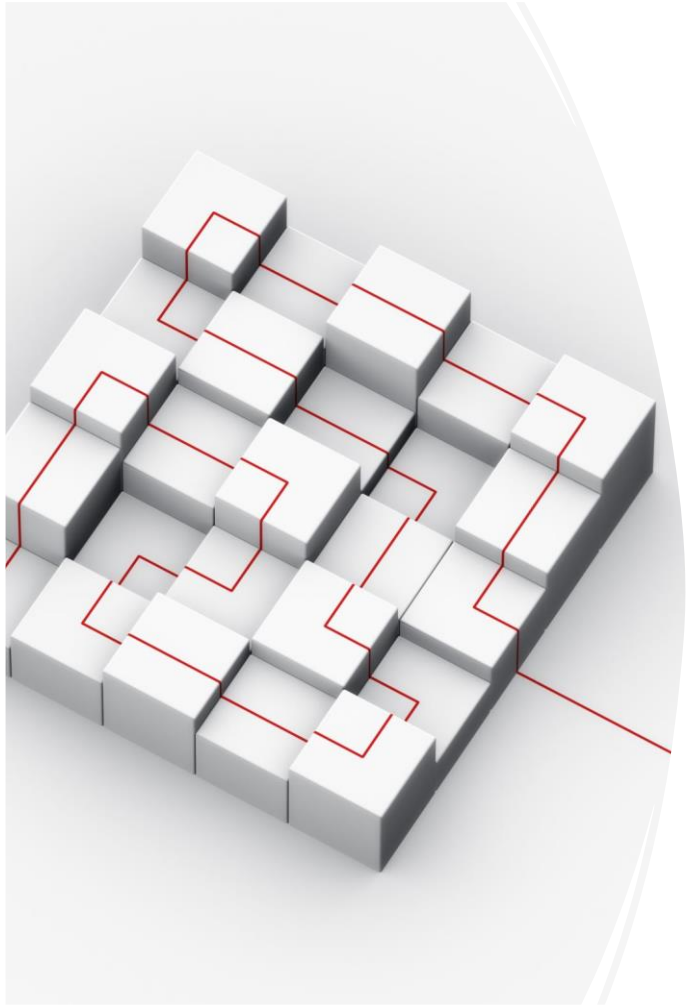
INT 161

Basic Backend Development

RESTful API
Exception Handling

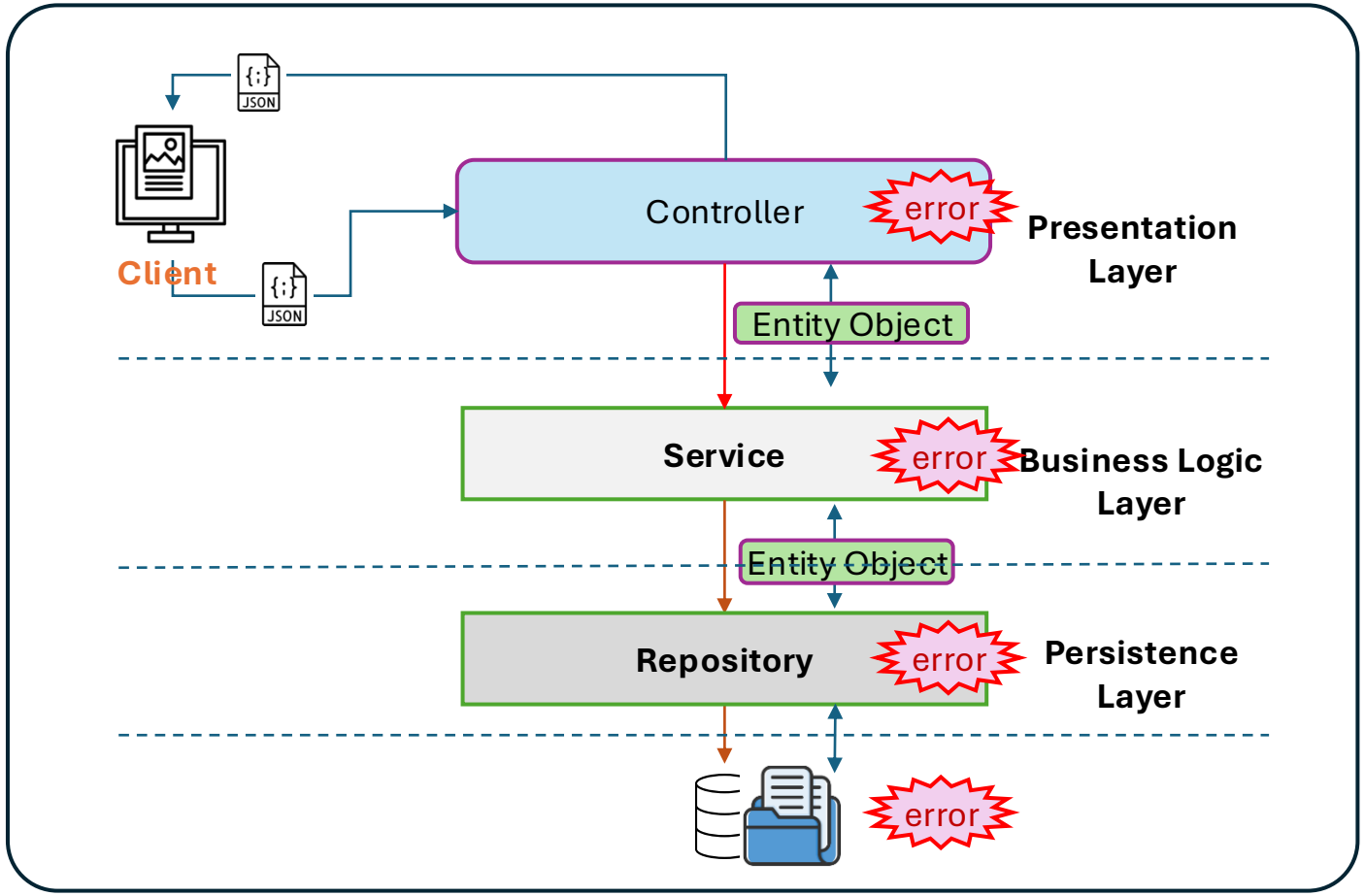




Unit Objectives

- After completing this unit, you should be able to:
  - Learn error handling using Express middle-ware
  - Design global error handling in Node/Express with best practices

Layered System



The **Layered System** principle means that a REST API is designed as a set of layers, where each layer has a specific role, and a client does not need to know whether it's communicating directly with the end server or through intermediaries.

This allows **scalability, flexibility, and separation of concerns**.

What is an Exception/Error?

- An exception/error is an event or condition that disrupts the normal flow of execution in a program.
- When an exception is thrown and not caught, Node.js terminates the process (process.exit(1)), causing the entire backend server to crash.
- Error is object
  - It encapsulates unexpected conditions in an object

Why Error Handling Matters

- Prevent app crashes / downtime
- Improve user experience (friendly error messages)
- Protect sensitive data from leaking (stack traces)
- Support debugging, monitoring, and alerting
- Error Types
  - Programming Errors → bugs, logic issues (TypeError, invalid assumptions)
  - Operational Errors → network timeouts, DB unavailable, file not found
  - Expected vs Unexpected Errors → handle expected gracefully

Error propagation

- Error propagation is a mechanism that allows errors to be passed up the call stack until they are caught and handled.
- This is crucial for debugging and ensuring that errors do not cause the entire application to crash unexpectedly.
- How errors propagate
  - When an error occurs in a function, it can either be caught and handled within that function or propagate up the call stack to the calling function.
  - If the calling function does not handle the error, it continues to propagate up the stack until it reaches the global scope, potentially causing the program to terminate.

Error Handling (Sync) : try .. catch

```
import fs from 'node:fs';

function main(value=0) {
    console.log(a(value));
}
function a(value) {
    return b(value);
}
function b(value) {
    if (value > 0) {
        return ++value;
    }
    const data = fs.readFileSync('test.txt');
}

main(); //without error handler
```

```
try {
    main(1);
    console.log('After main success');
    console.log('-----------------');
    main();
    console.log('After main error');
} catch (e) {
    console.log('Message: ', e.message);
    console.log('Status: ', e.status);
    console.log('Code: ', e.code);
    console.log('Stack Trace: ', e.stack);
}

console.log('Program was normal ended');
```

The standard JavaScript Error object

- message: A human-readable description of the error.
- name: The name of the error type (e.g., "Error", "TypeError", "ReferenceError").
- stack: (Non-standard but widely supported) A string representing the stack trace, indicating where the error occurred in the code.
- Additionally, when dealing with system errors in Node.js
  - code: A string representing the system error code (e.g., "EACCES" for permission denied).
  - errno: A number corresponding to the negated system error code.

Customer Error

```
  :
  :
async function b(value) {
    if (value > 0) {
        return ++value;
    }
    try {
        const data = await fs.readFile('data.txt', 'utf8');
        return data;
    } catch (e) {
        const err =  new Error('File data.txt not found');
        err.code = 'FILE_NOT_FOUND';
        err.status = 404;
        throw err;
    }
}
```

Custom Error Classes:

- For more specific error handling, you can create custom error classes (e.g., AppError) to categorize and add more context to errors.
- This can help in providing more informative error responses to clients and in internal logging.

```
class AppError extends Error {
   errors = {404:'NOT_FOUND', 400:'BAD REQUEST', 409:'CONFLICT',
   500:'INTERNAL SERVER ERROR', 403:'FORBIDDEN', 401:'UNAUTHORIZED'}
   constructor(message, statusCode) {
       super(message);
       this.statusCode = statusCode;
       this.error = this.errors[statusCode];
       this.isOperational = true;
       Error.captureStackTrace(this, this.constructor);
   }
}
```

Custom Error Classes example:

```
import fs from 'node:fs'

function main(value=0) {
  console.log(a(value));
}
function a(value) {
  return b(value);
}
function b(value) {
  if (value > 0) {
    return ++value;
  }
  const data = fs.readFileSync('test.txt')
}
```

```
// main(); //without error handler
try {
  main(1);
  console.log('After main success');
  console.log('------------------');
  main();
  console.log('After main error');
} catch (e) {
  const err = new AppError(e.message, 404)
  console.log(err);
}

console.log('Program was normal ended');
```

Express Middleware

Client (Request)
↓
Express.js Server (http)
↓
[Middleware 1] → next()
↓
[Middleware 2] → next()
↓
[Router Match] → handler
↓
[Response] → res.send() / res.json()
↓
Client (Response)

- middleware คือฟังก์ชันที่รับ (req, res, next)
- สามารถทำงานบางอย่าง เช่น log, parse body, auth, validate
- ถ้าเรียก next() ⟶ ส่งต่อไป middleware ถัดไป
- ถ้า res.send() หรือ res.end() ⟶ end (response to client)

```
// Middleware 1 - Logger
app.use((req, res, next) => {
    console.log(`${req.method} ${req.url}`)
    next()
})

//Middleware 2 - JSON parser
app.use(express.json())

app.use('/films', filmRouter);
```

Error Handling Mechanism in Express.js

- Express.js has a special mechanism to deal with errors using error-handling middleware.
  - Error Occurs
    - Your code (throw new Error, rejected Promise, etc.)
    - Middleware/Route failures (e.g., DB query error, invalid input)
  - Forward Error with next(err)
    - In Express, if you call next(err), Express will skip all normal middleware/route handlers and jump directly to an error-handling middleware.
  - Error Handling Middleware
    - Normal middleware signature: (req, res, next)
    - **Error-handling middleware signature: (err, req, res, next)**
    - These functions are used to catch and respond to errors.
  - Send Response Back, Typically, you send an appropriate HTTP status code and an error message/JSON.
    - 400 Bad Request
    - 401 Unauthorized
    - 500 Internal Server Error

Handling Errors

- The first step in handling errors is to provide a client with a proper **status code**. Additionally, we may need to provide more information in the response body.
- Basic Responses
  - The simplest way we handle errors is to respond with an appropriate status code.
  - Here are some common response codes:
    - 400 Bad Request - client sent an invalid request, such as lacking required request body or parameter
    - 401 Unauthorized - client failed to authenticate with the server
    - 403 Forbidden - client authenticated but does not have permission to access the requested resource
    - 404 Not Found - the requested resource does not exist
    - 409 Conflict - the request is conflict with the current state of the target resource.
    - 412 Precondition Failed - one or more conditions in the request header fields evaluated to false
    - 500 Internal Server Error - a generic error occurred on the server
    - 503 Service Unavailable - the requested service is not available

Express: Error-handling Middleware

```
// error handler
app.use(function (err, req, res, next) {
    const status = err.status || 500;
    res.status(status);
    res.json( {
            error: err.code,
            statusCode: status,
            message: err.message,
            path: req.originalUrl,
            timestamp: new Date().toLocaleString()
        }
    );
});
```

- Error-handling middleware must be defined **at the end**, after all routes/middlewares.
- If no error handler is defined, Express will send a default HTML error response (with stack trace in dev mode).

Practices

